

super keyword is used to call the parent constructor and its methods right



Yes, you are correct! The `super` keyword in Java is used primarily for two purposes:

1. To Call the Parent Class Constructor:

- The `super()` call is used to invoke the constructor of the parent class.
- It must be the first statement in the child class constructor if used explicitly.
- If `super()` is not explicitly called, Java automatically inserts a call to the no-argument constructor of the parent class.

Example:

```
java

class Parent {
    public Parent() {
        System.out.println("Parent constructor called");
    }
}

class Child extends Parent {
    public Child() {
        super(); // Calls the Parent class constructor
        System.out.println("Child constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
    }
}
```

Output:

```
kotlin

Parent constructor called
Child constructor called
```

2. To Access Parent Class Methods and Variables:

- The `super` keyword can also be used to call a method or access a variable from the parent class when it is shadowed by a child class method or variable.

Example:

java

```
class Parent {
    public void display() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    public void display() {
        super.display(); // Calls the Parent class method
        System.out.println("Child class method");
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}
```

Output:

kotlin

```
Parent class method
Child class method
```

Key Points About `super`:

1. `super()` for Constructor:

- Must be the first statement in the constructor.
- Calls the immediate parent class constructor.

2. `super` for Methods and Variables:

- Can be used anywhere (not restricted to the first line).
- Explicitly calls a method or accesses a variable in the parent class, avoiding shadowing by the child class.

3. No `super()` Needed If No Parent Constructor Exists:

- If the parent class has no constructor (or only has a no-argument constructor), `super()` is added implicitly by Java.

When Is `super` Necessary?

- To initialize a parent class when the parent has parameterized constructors.
- To avoid ambiguity when a child class overrides methods or hides variables from the parent class.



*SuperKeyword.java X

```
1 package Super;
2 class parentclass
3 {
4     public parentclass()
5     {
6         System.out.println("this is constructor");
7     }
8     public void methods()
9     {
10        System.out.println("this is method");
11    }
12 }
13
14 public class SuperKeyword extends parentclass
15 {
16     public SuperKeyword()
17     {
18         super(); //To call a parent cons, you need to need super() in child cons
19         super.methods();
20     }
21
22     public void method()
23     {
24         // super(); if you want to call parent cons, you need to use super in child cons
25         super.methods();
26         System.out.println("this is child class");
27     }
28
29
30
31     public static void main(String[] args)
32     {
33         SuperKeyword obj =new SuperKeyword();
34         obj.method();
35     }
36 }
```

Console X

```
<terminated> SuperKeyword [Java Application] C:\Users\user\p2\pc
this is constructor
this is method
this is method
this is child class
```

Activate Windows
Go to Settings to activate Windows.

Writable

Smart Insert

34 : 22 : 666